

SPIRIT

ICD - V.2.4.0

U-Plan Preparation Service (UPPS)

Álex Sanchis

May 14, 2026

Resumen

Este documento describe la(s) interfaz(ces) del sistema **UPPS** (parte de **U-Plan Preparation Service, UPPS**) dentro del contexto del proyecto **SPIRIT**. En particular, especifica:

- La interfaz entre el **cliente web** (operador UAS) y el **backend UPPS** (API HTTP de Next.js).
- La integración del backend con servicios externos: **FAS** (*Flight Authorization Service*) y **Geoawareness Service**.
- La integración con el **Geoawareness Service** mediante un **proxy SSE** en el backend para obtención de geozonas y datos de U-space en tiempo real (sin conexión directa desde el navegador).
- La coordinación del procesamiento distribuido de trayectorias mediante un **daemon de asignación** (`traj-assigner.js`) y **máquinas de proceso** (workers) que ejecutan el proyecto externo `traj-runner`.
- El sistema de **regeneración automática de volúmenes de operación** para planes procesados.
- El **workflow guiado de 5 pasos** para preparación, procesamiento y autorización de planes de vuelo en la página **Plan Authorization**.
- Generación interactiva de planes de vuelo en la página **Plan Definition**.
- Sistema de **visualización interactiva** con mapas Leaflet para trayectorias, waypoints y geozonas.

El objetivo de la interfaz es permitir que un usuario cargue planes de vuelo (típicamente exportados desde QGroundControl o generados directamente en la aplicación), que dichos planes se procesen para generar trayectorias (CSV) y, finalmente, que esas trayectorias puedan convertirse a **U-Plan** para su evaluación de geoawareness y envío al servicio de autorización (FAS).

Novedades en V2.0.0 SPIRIT

- **Mejora general UI/UX:** Nuevo esquema visual, mejora de experiencia de usuario, optimización para escritorio y móvil.
- **Sistema de temas dinámico:** Soporte completo para temas claro/oscuro con cambio instantáneo y persistencia en localStorage.
- **Normalización de tiempos:** Las trayectorias CSV se normalizan automáticamente para iniciar en $t=0s$, independientemente del tiempo inicial de simulación.
- **Regeneración automática de volúmenes:** Sistema que verifica cada 30 segundos planes procesados sin volúmenes y los regenera automáticamente.
- **Optimización del flujo de volúmenes:** Eliminada la regeneración redundante de volúmenes antes del envío a FAS cuando ya existen, reduciendo notificaciones duplicadas y carga de procesamiento.
- **Viewer de trayectorias:** Visualización interactiva de trayectorias con mapas Leaflet, sustituyendo la descarga directa de CSV.
- **Visualización mejorada de volúmenes de operación:**
 - Todos los volúmenes se muestran simultáneamente en el mapa (no solo los activos).
 - Volúmenes activos destacados en morado brillante (40% opacidad).
 - Volúmenes inactivos mostrados en gris semi-transparente (15% opacidad) para contexto.
 - Control de tiempo permite navegar entre ventanas temporales.
 - Carga fresca de datos desde API antes de cada visualización para garantizar sincronización.
- **Geoawareness mejorado:** Auto-centrado de mapas en trayectorias, visualización con marcadores reducidos y overlays optimizados.
- **Gestión avanzada de folders:** Drag-and-drop para organización de planes, sección de planes huérfanos.
- **Timestamps dinámicos en U-Plan:** Los campos `creationTime` y `updateTime` se actualizan automáticamente al momento del envío a FAS.
- **Workflow optimizado:** Interfaz guiada de 5 pasos con indicadores de progreso, bloqueo de edición post-procesamiento y validaciones mejoradas.
- **Arquitectura modular mejorada:** Componentes reutilizables organizados por funcionalidad con hooks especializados.

Novedades en V2.2.0

Sprint 1 (post v2.1.0) — Robustez, UX y mejoras de flujo

- **Eliminación de validación DNS en registro** (Feature 1): Se elimina la función `validateEmailDomain()` del endpoint de registro, que bloqueaba dominios corporativos con configuraciones DNS no estándar. Solo se mantiene la validación de formato Zod.
- **Caducidad del código de verificación reducida a 3 minutos** (Feature 2): Código de verificación de email válido 3 minutos (antes 24 h); doble-verificación `safe-guard` (`emailVerified: false`); mensaje de cooldown corregido en `/api/auth/resend-verify`.
- **Fix tema del sistema y toasts** (Feature 7): Tema por defecto basado en preferencia del SO; toasts totalmente opacos en ambos temas.
- **Validación de archivo U-Plan en carga** (Feature 8): Toast de error detallado si el archivo tiene extensión incorrecta, no es JSON válido, o carece de `operationVolumes`.
- **Campos de perfil de usuario** (Feature 9): Nuevos campos en BD `firstName`, `lastName`, `phone`; nuevos endpoints `GET/PATCH /api/user/profile`; `UplanFormModal` se pre-rellena automáticamente con los datos del perfil.
- **Notificación por email al cancelar plan aprobado** (Feature 10): Se envía email al usuario cuando se elimina un plan con `authorizationStatus="aprobado"`.
- **Botón OK y tecla Enter en edición de waypoints** (Feature 4): Los popups de edición de lat/lon incluyen botón OK y soporte de Enter para confirmar sin cerrar el teclado.
- **Botón “Añadir Waypoint” en la barra lateral** (Feature 5): Formulario inline para introducir coordenadas manualmente sin hacer click en el mapa.
- **Prevención de zoom reset en transiciones SCAN** (Feature 3): Se elimina `scanHandlerVersion` de la key de `PlanMap`, evitando el remontaje del mapa Leaflet en cada paso del flujo SCAN.
- **Generación automática de volúmenes antes de descargar U-Plan** (Feature 6): Si el U-Plan no tiene volúmenes de operación, se generan automáticamente antes de lanzar la descarga.
- **Visualización de denegación mejorada — coloreado selectivo** (Feature 12): Solo los volúmenes geoméricamente conflictivos se colorean en rojo (intersección 2D de polígonos); se muestra la razón FAS parseada con número de volúmenes y geozonas conflictivos.
- **Restricciones regulatorias EU para clase UAS** (Feature 13): Tabla de restricciones C0–C6 (MTOM máximo y dimensión máxima) validada en `PlanGenerator` y `UplanFormModal`.
- **Visor 3D de volúmenes de operación U-Plan con CesiumJS** (Feature 11): Nuevo componente `Cesium3DModal` con globe OSM Buildings; botón “View 3D” en `FlightPlansUploader`; nuevo endpoint `GET /api/cesium/token`.

Sprint 2 — Nuevas Funcionalidades (visualización 3D, trayectorias, contacto)

- **Edición de perfil en /profile** (Feature 14): La edición de `firstName`, `lastName` y `phone` se traslada de `/settings` a `/profile`; la página de ajustes conserva solo tema, idioma y notificaciones.
- **Corrección de altitud AGL en trayectorias** (Feature 15): Al convertir trayectoria CSV a U-Plan se resta `plannedHomePosition[2]` de todos los valores `Alt`; los visores 3D usan `RELATIVE_TO_GROUND`.
- **Slider 4D en el visor 3D U-Plan** (Feature 16): Barra de tiempo con play/pause en `Cesium3DModal`; volúmenes activos resaltados, inactivos atenuados; velocidades 1×, 2×, 5×, 10×.
- **Modal unificado de resultado de autorización FAS 2D/3D** (Feature 18): Nuevo componente `AuthorizationResultModal` con pestañas 2D/3D; muestra planes aprobados y denegados; sección “Parsed FAS Reason” en lenguaje natural.
- **Mapa 3D de denegación con volúmenes de geozonas** (Feature 17): Nuevo componente `Denial3DModal`; volúmenes operativos en gris/rojo más prismas 3D semitransparentes de geozonas conflictivas con sus límites verticales.
- **Página de Contacto con sistema de tickets** (Feature 19): Formulario con Asunto, Categoría y Descripción; nuevo endpoint `POST /api/contact` que genera número de ticket `UPPS-AAAAMMDD-XXXX`; email de confirmación al usuario y notificación al equipo de soporte.
- **Visor 3D de trayectoria con dron animado** (Feature 20): Nuevo componente `Trajectory3DViewer` (Cesium); trayectoria como polilínea 3D semitransparente; slider temporal + play/pause; modelo de dron interpolado sobre la trayectoria.

Sprint 3 — Polish, correcciones y arquitectura

- **Renombrado de páginas principales:**
 - `/plan-generator` renombrado a `/plan-definition` (*Plan Definition*).
 - `/trajectory-generator` renombrado a `/plan-authorization` (*Plan Authorization*).
 - Los componentes internos `plan-generator/` han pasado a `plan-definition/`.
- **Navegación directa Plan Definition → Plan Authorization** (Sprint 3 #21):
 - Tras subir un plan desde Plan Definition, la app navega automáticamente a `/plan-authorization?selectPlan=<id>`.
 - Plan Authorization lee el parámetro `selectPlan` y auto-selecciona el plan recién importado.
- **Sincronización del campo state del U-Plan con FAS** (Sprint 3 #22):
 - El handler `PUT /api/fas/<externalResponseNumber>` actualiza ahora el campo `state` dentro del JSON del U-Plan almacenado en BD, reflejando el estado comunicado por FAS (`ACCEPTED`, `WITHDRAWN`, etc.).

- **Corrección de tema oscuro en InfoBox de Cesium** (Sprint 3 #23):
 - Se inyecta CSS en el iframe de `viewer.infoBox.frame` para forzar fondo oscuro y texto claro, eliminando el problema de texto blanco sobre fondo blanco en tema oscuro.
 - Aplicado en: `Cesium3DModal`, `AuthorizationResultModal`, `Denial3DModal`, `Trajectory3DViewer`.
- **Corrección del visor 3D de trayectoria** (Sprint 3 #24):
 - El dron ahora se actualiza correctamente durante la reproducción (usando `ref` en lugar de estado `React` para la lógica del `RAF tick`).
 - Unificación de referencia de altura: `trayectoria`, `waypoints` y `dron` usan `RELATIVE_TO_GROUND` (AGL) de forma consistente, eliminando la doble ruta visible.
- **Proxy SSE para Geoawareness en el backend** (Sprint 3 #25 + mejora posterior):
 - Arquitectura cambiada: el **navegador ya no abre WebSocket directamente** al servicio `Geoawareness`.
 - El backend (`Next.js`) abre la conexión `ws://` internamente y retransmite cada mensaje al navegador como un evento SSE (*Server-Sent Events*) via `GET /api/geoawareness`
 - Esto elimina los errores de *mixed-content* en despliegues HTTPS sin requerir TLS en el servicio `Geoawareness`.
 - Solo se requiere la variable de entorno `servidor GEOAWARENESS_SERVICE_IP`; se deprecia `NEXT_PUBLIC_GEOAWARENESS_SERVICE_IP`.
- **Tooltips de ayuda en el formulario U-Plan** (Sprint 3 #26):
 - Nuevo componente reusable `FieldHelp` (icono `?`) con tooltip al hacer click.
 - Explica acrónimos de aviación: SAC, SIC, MTOM, VLOS, BVLOS, SAIL, etc.
 - Añadido en `UplanFormModal` y `PlanGenerator/PlanDefinition`.
- **Corrección de activos Cesium en servidor** (fix buildtime):
 - El script `postinstall` y el nuevo hook `prebuild` copian **todo** el directorio `Build/Cesium/` (incluyendo `Cesium.js`) a `public/cesium/`, corrigiendo el error 404 en despliegues donde `public/cesium` está en `.gitignore`.

Sprint 4 — Bug Fixes, UX Polish & Nuevas Funcionalidades

- **Fix popup waypoint manual — OK cierra popup, Enter dispara OK** (Feature 27): El botón OK en popups de edición de waypoints ahora cierra el popup tras confirmar cambios. La tecla Enter dispara la misma lógica que OK. Aplicado también a todos los popups `SCAN` (takeoff, landing, vértices polígono).
- **Mejoras SCAN pattern — Add Vertex, bounds efectivos, validación drag** (Feature 28): Nuevo botón “Add Vertex” en paso 2 del sidebar `SCAN`; los límites del servicio usan `effectiveServiceLimits` (U-space seleccionado) en lugar de constantes globales; la operación de arrastrar vértices, takeoff y landing se valida contra los bounds del servicio.

- **Segundo click en plan seleccionado abre modal de waypoints** (Feature 29): Al hacer click en un plan ya seleccionado, se abre el modal de previsualización de waypoints en lugar de deseleccionar el plan.
- **Fix actualización de color de volúmenes 4D durante Play** (Feature 30): Se reemplaza la asignación directa de `material` y `outlineColor` en entidades Cesium por `ColorMaterialProperty` y `ConstantProperty`, lo que permite que Cesium detecte el cambio y re-renderice correctamente durante la animación.
- **Fix polilínea de trayectoria 3D — eliminado terreno AMSL** (Feature 31): Se elimina el muestreo de terreno (`sampleTerrainMostDetailed`) del visor `Trajectory3DViewer`; la polilínea usa directamente altitudes AGL del CSV, unificando la referencia de altura con waypoints y dron.
- **Visor 3D de Geoawareness con geozonas extruidas** (Feature 32): Nuevo componente `Geoawareness3DModal` basado en CesiumJS que muestra geozonas como polígonos 3D extruidos coloreados por tipo de restricción (`GEOZONE.COLORS`), trayectoria como polilínea azul y volúmenes de operación. Las geozonas son clickables y muestran información de autoridad, restricciones y aplicabilidad en un `InfoBox`. Modal de Geoawareness 2D ampliado (85vw/1600px). Botón “3D Geoawareness” en la barra de herramientas de Plan Authorization.
- **Auto-relleno de email en formulario U-Plan** (Feature 33): El campo `contactDetails.email` del formulario U-Plan se pre-rellena automáticamente con el email del perfil del usuario activo.
- **Sistema de borradores U-Plan** (Feature 34): Nuevo modelo Prisma `uplanDraft` para persistir borradores de U-Plan vinculados al usuario. Endpoints CRUD completos: `GET/POST /api/user/drafts` y `GET/PATCH/DELETE /api/user/drafts/[id]`. Sección “U-Plan Drafts” en la página `/profile` para listar, crear, editar nombre y eliminar borradores. El formulario `UplanFormModal` incluye desplegable “Load Draft” y botones “Save as Draft” / “Update Draft”.
- **Eliminación de sección Flight Plan Details de Plan Definition** (Feature 35): Se eliminan ~840 líneas del componente `PlanGenerator` (estado `flightPlanDetails`, botón “Flight Plan Details” y todo su bloque de contenido), simplificando el flujo de definición de plan.

Sprint 5 — V2.2.1 Bug Fixes, UX & Nuevas Funcionalidades

- **Fix overflow de tooltips en formulario U-Plan** (Task 1): Los tooltips del formulario ahora se renderizan mediante un portal, evitando el recorte por contenedores con `overflow:hidden`.
- **Respuestas FAS 400 establecen estado “withdrawn”** (Task 2): Si FAS responde con HTTP 400, el U-Plan pasa a estado `withdrawn` y se envía email de denegación al usuario.
- **Centrado de mapa en waypoints manuales** (Task 3): Al añadir un waypoint mediante coordenadas manuales en Plan Definition, el mapa se centra automáticamente en la nueva posición.

- **Fix cámara de seguimiento y proximidad de etiquetas en visor 3D** (Task 4): Corregida la cámara de seguimiento del dron y la distancia de las etiquetas de waypoints en Trajectory3DViewer.
- **Timeout de inactividad de 10 minutos** (Task 5): Se añade un overlay de “Sesión expirada” tras 10 minutos de inactividad del usuario, requiriendo re-login.
- **Eliminación del botón auto-play en visor 3D de U-Plan** (Task 6): Se elimina el botón de reproducción automática del visor 3D de línea temporal de U-Plan.
- **Restricción de fechas pasadas en date picker** (Task 7): El selector de fecha de programación de vuelo impide seleccionar fechas anteriores al momento actual.
- **Reset de plan aprobado cancela autorización FAS** (Task 8): Resetear un plan aprobado envía cancelación a FAS y notificación por email al usuario, con modal de confirmación.
- **Bloqueo de reset para U-Plans externos** (Task 9): Los U-Plans subidos externamente no pueden resetearse; se muestra mensaje informativo.
- **U-Plans externos omiten recompilación de volúmenes** (Task 10): Al pulsar “Save & Request Auth” en un U-Plan externo se envía directamente a FAS sin regenerar volúmenes.
- **Fix overflow vertical en popup de geozonas** (Task 11): El popup de información de geozonas en Plan Definition ahora es scrollable verticalmente, evitando desbordamiento.
- **Fix renderizado 3D de geozonas** (Task 12): Corregido el renderizado de geozonas 3D en los modales de Geoawareness y Resultado de Autorización.
- **Eliminación de adjuntos JSON en emails de autorización** (Task 13): Los emails de resultado de autorización ya no incluyen el JSON del U-Plan como adjunto.
- **Bump de versión y actualización de documentación** (Task 14): Versión actualizada a 2.2.1; documentación ICD actualizada con changelog del Sprint 5.
- **Nueva splash page AETHON con shader animado** (Task 15): Nueva página de bienvenida con fondo de shader animado WebGL y enlace a /launch.

Novedades en V2.1.0

- **Sistema de gestión de emails con MailerSend:**
 - Verificación de email con código de 6 dígitos y link de activación (24h validez).
 - Reset de contraseña con token seguro (1h validez).
 - Notificaciones automáticas de resultado de autorización FAS con UPLAN adjunto.
 - Configuración mediante variables de entorno: `MAILERSEND_API_KEY`, `MAILERSEND_SENDER_EMAIL`
- **Importación de UPLANs externos:**

- Drag-and-drop de archivos .json con formato UPLAN directamente en carpetas.
- Botón de upload visible con icono de subida (verde) en cabecera de carpetas.
- Validación automática de estructura UPLAN (operationVolumes requeridos).
- Sin regeneración de volúmenes: UPLANs externos se envían directamente al FAS.
- Detección automática basada en ausencia de `csvResult`.
- **Selector de U-Space para UPLANs externos:**
 - Modal interactivo con mapa Leaflet mostrando todos los U-Spaces disponibles.
 - Click en mapa selecciona el U-Space más pequeño que contiene el punto.
 - Visualización de polígonos con colores: azul (disponible), naranja (seleccionado).
 - Tooltip con nombre del U-Space al pasar el ratón.
 - Asignación automática de `uspace_identifier` en `geoawarenessData`.
- **Visualización de denegaciones en mapa:**
 - Modal específico para mostrar por qué un plan fue denegado.
 - Volúmenes conflictivos en rojo, volúmenes válidos en verde.
 - Geozonas conflictivas resaltadas en el mapa.
 - Parsing robusto de múltiples formatos de respuesta FAS:
 - * Formato A: objetos con `{ordinal, status}`.
 - * Formato B: arrays de índices numéricos directos.
 - * Formato C: sin campo volumes, todos los volúmenes marcados como conflictivos.
 - Botón "View Denial on Map" visible cuando `authorizationStatus="denegado"`.
- **Edición de waypoints en SCAN patterns:**
 - Popups editables en puntos de takeoff, landing y vértices del polígono.
 - Campos de input para latitud y longitud con actualización en tiempo real.
 - Estilos compatibles con tema claro/oscuro usando CSS variables.
 - Actualización bidireccional: drag-and-drop o edición manual de coordenadas.
- **Mejoras en tema oscuro:**
 - Popups de Leaflet con estilos globales para tema oscuro/claro.
 - `ScanPatternGeneratorV2` completamente adaptado con CSS variables (40+ reemplazos).
 - Eliminación de clases hardcoded `bg-zinc-*`, `text-zinc-*`, `border-zinc-*`.
 - Uso consistente de `var(--bg-primary)`, `var(--text-primary)`, etc.
- **Botón de descarga de U-Plan:**
 - Botón "Download U-Plan" visible después de procesamiento si existe `uplan`.

- Descarga directa del JSON con nombre `uplan_<planname>.json`.
- Ubicado junto a los botones de acción (Process, Geoawareness, Authorize).
- **Manejo robusto de errores 500 del FAS:**
 - Sistema de reintentos automático: hasta 5 intentos con backoff exponencial (1s, 2s, 4s, 8s, 16s).
 - Diferenciación de errores: 500 (reintento), 400/404 (marcar como denegado).
 - Si todos los reintentos fallan con 500: plan vuelve a estado "sin autorización".
 - Respuesta HTTP 503 (Service Unavailable) al cliente para distinguir de errores reales.
 - Toast informativo: "FAS temporarily unavailable. Please try again later."
- **Correcciones de formato para UPLANs externos:**
 - `scheduledAt` convertido a ISO-8601 con timezone (añade 'Z' si falta).
 - Validación de `timeBegin` en `operationVolumes` antes de inserción en BD.
 - Prevención de errores de Prisma `DateTime` al importar UPLANs externos.
- **Mejora en flujo de selección de U-Space:**
 - Apertura inmediata del viewer de geoawareness tras selección (sin segundo click).
 - Llamada directa a API de geoawareness sin depender de actualización de estado React.
 - Reducción de timing issues en flujo de usuario.

1 Alcance de la Interfaz

- **Cliente web (navegador):** interfaz React/Next.js utilizada por el operador para cargar planes, organizar carpetas, lanzar procesamiento, descargar CSV y gestionar el envío a autorización.
- **Backend UPPS (Next.js API):** endpoints HTTP bajo `/api/*` que realizan operaciones CRUD sobre planes, carpetas y máquinas, y exponen endpoints especializados para **U-Plan** y **geoawareness**.
- **Base de datos MySQL:** persistencia de usuarios, carpetas, planes, resultados CSV y estado de máquinas (gestionada con Prisma).
- **Daemon de asignación de trabajos:** `traj-assigner.js` (Node.js) asigna planes en cola a máquinas disponibles mediante actualización de registros en BD.
- **Workers de procesamiento:** máquinas/VMs externas (no incluidas en este repositorio) que ejecutan `traj-runner` y escriben resultados en BD (tabla `csvResult`) y estado del plan.
- **Servicios externos:**

- **FAS** (Flight Authorization Service): recibe el U-Plan generado y posteriormente notifica el resultado al UPPS.
- **Geoawareness Service**: proporciona datos de U-spaces y geozonas. El backend UPPS abre una conexión WebSocket interna a `ws://<GEOAWARENESS_SERVICE_IP>/ws/ga` y retransmite los mensajes al cliente web como eventos SSE a través de `GET /api/geoawareness/stream/{uspaceId}`. El navegador nunca se conecta directamente al servicio Geoawareness.
- (Opcional) **noVNC/QGroundControl**: el frontend incluye un componente que embebe un `iframe` apuntando a `http://localhost:6080/` para visualizar/controlar una instancia VNC de QGroundControl.

Fuera de alcance: telemetría en tiempo real, control de aeronaves, enlaces MAVLink, o envío formal a autoridades (objetivos futuros descritos en `README.md`, no implementados completamente en el estado actual del repositorio).

2 Entradas y Salidas

Entradas

Las entradas al sistema UPPS se reciben principalmente como solicitudes HTTP (JSON) al backend, así como respuestas/callbacks de servicios externos y actualizaciones en base de datos realizadas por procesos internos.

Entradas desde sistemas externos (FAS y Geoawareness)

Campo	Tipo de dato	Formato	Descripción
FAS callback: endpoint	String	Ruta HTTP	Callback de FAS hacia UPPS: <code>PUT /api/fas/<externalResponseNumber></code> .
FAS callback: state	String	Texto	Campo <code>state</code> recibido en el callback. Si <code>state=ACCEPTED</code> se marca <code>authorizationStatus="aprobado"</code> ; en caso contrario "denegado".
FAS callback: message	String/Null	Texto	Campo <code>message</code> recibido en el callback; se persiste como <code>authorizationMessage</code> .
Geoawareness SSE: endpoint	String	URL HTTP	El cliente web se conecta al proxy del backend: <code>GET /api/geoawareness/stream/{USPACE_ID}</code> . El backend abre internamente <code>ws://<IP>/ws/gas/{USPACE_ID}</code> y retransmite mensajes como SSE.

Campo	Tipo de dato	Formato	Descripción
Geoawareness SSE: message	Object	JSON	Evento SSE (tipo data) con el mensaje JSON del servicio Geoawareness. Estructura de 3 bloques: <code>uspace_identifier</code> , <code>uspace_data</code> , y <code>geozones</code> (FeatureCollection). Ver sección ??.

Entradas desde sistemas internos (cliente web, procesos internos y BD)

Campo	Tipo de dato	Formato	Descripción
userId	Integer	id BD	Identificador del usuario (parámetros/relación). Por ejemplo: <code>GET /api/flightPlans?userId=...</code> y <code>GET /api/folders?userId=...</code>
folderId	Integer/Null	id BD	Carpeta destino de un plan (opcional en creación/actualización).
customName	String	UTF-8	Nombre del plan. En la UI suele derivar del nombre de fichero sin extensión.
fileContent	String	Texto	Contenido del plan cargado desde QGroundControl (enviado como texto). Se guarda en <code>flightPlan.fileContent</code> .
status	String	Enumerado	Estado de plan: sin procesar , en cola , procesando , procesado , error .
scheduledAt	String/Null	ISO-8601	Fecha/hora programada del vuelo. Obligatoria para geoawareness y generación/envío de U-Plan.
uplan	Object/String	JSON	Detalles de U-Plan aportados por el usuario (opcional). Se utiliza para rellenar campos del U-Plan final.
authorizationStatus	String	Enumerado	Estado local de autorización (persistido en BD): sin autorización (por defecto), aprobado , denegado .
authorizationMessage	String/Object	Texto/JSON	Mensaje asociado a la autorización (errores, feedback, etc.).
externalResponseNumber	String/Null	Texto	Identificador devuelto por FAS tras envío del U-Plan; se usa para correlación del callback.

Campo	Tipo de dato	Formato	Descripción
geoawarenessData	Object/Null	JSON	Campo JSON en BD para almacenar datos de geoawareness asociados al plan.
Auth: email/password	String	UTF-8	Credenciales recibidas en POST /api/auth/signup y POST /api/auth/login.
Auth: Authorization header	String	Bearer JWT	Cabecera Authorization: Bearer <token> requerida en GET /api/user.

Salidas

Las salidas del sistema consisten en respuestas HTTP al cliente web y en peticiones HTTP a servicios externos (FAS y geoawareness).

Salidas hacia sistemas externos (FAS y Geoawareness)

Campo	Tipo de dato	Formato	Descripción
HTTP request a FAS	JSON	application/json	POST <FAS_API_URL> (si no se define FAS_ENDPOINT, se usa el path /uplan). Cuerpo: U-Plan. Cabecera: Content-Type: application/json.
SSE stream a cliente (proxy Geoawareness)	N/A	text/event-stream	El backend abre ws://<GEOAWARENESS_SERVICE_IP>/ws/g y retransmite cada mensaje al cliente via SSE (GET /api/geoawareness/stream/{uspaceId}). El navegador nunca abre WebSocket directo al servicio.

Salidas hacia sistemas internos (respuestas de la API al cliente web)

Campo	Tipo de dato	Formato	Descripción
FlightPlan	JSON	–	Objeto de plan devuelto por /api/flightPlans (incluye campos de BD; en listados incluye folder).
Folder	JSON	–	Objeto de carpeta devuelto por /api/folders.
Machine	JSON	–	Objeto de máquina devuelto por /api/machines.

Campo	Tipo de dato	Formato	Descripción
Bulk counters	Integer	–	<code>createdCount</code> (creación bulk) y <code>count</code> (actualización bulk) en <code>/api/flightPlans</code> .
CSV (individual)	String	CSV texto	GET <code>/api/csvResult?id=...</code> devuelve <code>{csvResult: "..."}.</code>
CSV bulk items	Array	JSON	POST <code>/api/csvResult</code> devuelve <code>{items: [{id, customName, csvResult}]}</code>
Delete report	JSON	–	DELETE <code>/api/flightPlans</code> devuelve conteos: <code>deletedPlans</code> , <code>deletedCsvs</code> , <code>totalDeleted</code> .
U-Plan generado	JSON	application/json	POST <code>/api/flightPlans/<id>/uplan</code> devuelve <code>{uplan, authorizationMessage}.</code>
Geoawareness result	JSON	–	POST <code>/api/flightPlans/<id>/geoawareness</code> devuelve <code>geozones</code> , <code>trajectory</code> , <code>uplan</code> , <code>hasConflicts</code> , <code>serviceAvailable</code> , <code>planName</code> .
Auth token	String	JWT	POST <code>/api/auth/login</code> devuelve <code>{token}.</code>

Formato CSV de trayectoria (mínimo requerido)

El conversor `trayToUplan` y el endpoint de geoawareness asumen un CSV con **encabezados** que incluya al menos las columnas: `SimTime`, `Lat`, `Lon`, `Alt`. Para geoawareness, la trayectoria para el mapa se extrae asumiendo que `Lat` y `Lon` están en las posiciones 2 y 3 (1-indexed) al separar por comas.

Formato U-Plan (salida generada, alto nivel)

El U-Plan se genera a partir del CSV y `scheduledAt` y contiene una lista de `operationVolumes` (polígonos GeoJSON con ventana temporal y límites verticales). Los campos adicionales (operador, UAS, categorías, etc.) se rellenan desde `flightPlan.uplan` si se proporciona; en caso contrario, se generan valores por defecto/aleatorios.

3 Protocolos y Transporte

- **Cliente web ↔ UPPS:** HTTP (Next.js). Puerto expuesto típicamente: 3000.
 - Rutas API (`pages/api/*`): REST-like sobre HTTP con payload JSON.
 - Límite de cuerpo (body parser): 50mb en `/api/flightPlans` y 10mb en `/api/csvResult`.
- **UPPS ↔ MySQL:** Prisma se conecta a MySQL mediante `DATABASE_URL`. En el ejemplo de configuración, MySQL usa el puerto 3306.

- **UPPS → FAS:** HTTP POST sin TLS a `http://<FAS_IP>/<FAS_ENDPOINT>`. Si `FAS_ENDPOINT` no existe, se usa el path `uplan`. Content-Type: `application/json`.
- **FAS → UPPS:** HTTP PUT a `/api/fas/<externalResponseNumber>` con JSON `{state, message}`.
- **Cliente web ↔ Geoawareness (SSE proxy):** La arquitectura utiliza SSE como capa de transporte entre el navegador y el backend, y WebSocket interno entre el backend y el servicio Geoawareness:
 - **Navegador → Backend (SSE):** GET `/api/geoawareness/stream/{uspaceId}`
— EventSource HTTP estándar, compatible con HTTPS sin restricciones de `mixed-content`.
 - **Backend → Geoawareness (WS interno):** `ws://<GEOAWARENESS_SERVICE_IP>/ws/gas/`
— conexión WebSocket en la red interna del servidor; no requiere TLS.
 - **Eventos SSE emitidos:** `connected` (WS abierto), `data` (mensaje del servicio), `error` (error WS), `close` (WS cerrado).
 - **Reconexión automática:** el hook `useGeoawarenessWebSocket` implementa backoff exponencial (1s, 2s, 4s, 8s, 16s) sobre la conexión SSE.
 - **Fallback:** Si la conexión SSE falla después de 5 intentos, el sistema carga datos estáticos de geozonas desde `/api/deprecated/geoawareness-geozones`.
- **(Deprecated) UPPS → Geoawareness HTTP:** El endpoint HTTP POST `/geozones\_searcher\_by\_volumes` está deprecado. Usar el proxy SSE para nuevas integraciones.
- **(Opcional) noVNC/QGroundControl:** la UI incluye un `iframe` a `http://localhost:6080/`. Este transporte depende de que noVNC esté levantado localmente.
- **Autenticación:**
 - POST `/api/auth/login` devuelve un JWT.
 - GET `/api/user` requiere `Authorization: Bearer <token>`.
 - Los endpoints bajo `pages/api/*` no aplican verificación JWT en el estado actual del repositorio.

4 Identificadores y Convenciones

- **IDs de recursos (BD):** todos los identificadores principales son enteros autoincrementales (`user.id`, `folder.id`, `flightPlan.id`, `machine.id`, `csvResult.id`).
- **Convención clave plan–CSV:** el borrado unificado asume `flightPlan.id == csvResult.id` (se eliminan CSVs con los mismos IDs que los planes borrados).
- **Correlación FAS:** `externalResponseNumber` (string) identifica el plan en el callback `/api/fas/<externalResponseNumber>`.
- **Estados de procesamiento:**
 - `flightPlan.status`: `sin procesar`, `en cola`, `procesando`, `procesado`, `error`.

- `machine.status`: Disponible, Ocupada.
- `flightPlan.authorizationStatus`: por defecto sin autorización; actualizaciones a aprobado o denegado tras callback FAS.
- **Unidades/formatos:**
 - `scheduledAt`: ISO-8601 (string) en API/BD.
 - CSV: lat/lon en grados decimales; altitud numérica.
 - U-Plan: geometrías GeoJSON con orden [lon, lat].
 - Altitudes en U-Plan: `uom="M"` y `reference="AGL"`.

5 Sincronización y Temporización

- **Asignación de planes a máquinas:** el proceso `traj-assigner.js` ejecuta asignación cada **10 ms** (`setInterval(assignNextPlan, 10)`).
- **Limpieza automática de CSV pequeños:** el mismo proceso ejecuta limpieza periódica cada **5 minutos** (`setInterval(cleanSmallCsvResults, 60 * 5000)`).
- **Regeneración automática de volúmenes:** el hook `useVolumeRegeneration` ejecuta verificación cada **30 segundos** para planes procesados sin volúmenes de operación.
- **Polling de estado de planes:** el hook `useFlightPlans` actualiza automáticamente el estado cada **5 segundos** cuando está habilitado.
- **Geoawareness WebSocket:**
 - Reconexión con backoff exponencial: 1s, 2s, 4s, 8s, 16s (máximo 5 intentos).
 - Fallback a HTTP después de agotar intentos.
- **Normalización de tiempos de trayectoria:**
 - Parser CSV normaliza todos los tiempos para iniciar en `t=0s`.
 - Se aplica en: `parseCSVToTrajectory` (GeoawarenessViewer, TrajectoryMapView).
 - Se aplica en: `trajToUplan` para generación de volúmenes con tiempos correctos.
- **Actualización de timestamps U-Plan:** `creationTime` y `updateTime` se actualizan al momento exacto del envío a FAS (ISO-8601 UTC).
- **JWT:** expiración del token de **1 día**.
- **Límites operacionales de la API (bulk):**
 - `/api/csvResult`: máximo 5000 IDs por request (POST/DELETE bulk).
 - `/api/flightPlans`: el endpoint acepta hasta 100000 IDs en actualizaciones/borrados bulk; las actualizaciones por-item se trocean en chunks de 200 operaciones por transacción.

6 Gestión de Errores

- **Códigos HTTP usados:**

- 200: operación correcta.
- 201: recurso creado.
- 204: borrado correcto sin contenido.
- 400: parámetros inválidos o campos requeridos ausentes.
- 401: autenticación requerida o token inválido (`/api/user`).
- 404: recurso no encontrado.
- 405: método no permitido.
- 500: error interno (BD/red/errores no controlados).
- 503 (v2.1): servicio temporalmente no disponible (FAS no responde después de 5 reintentos).

- **Errores en integración FAS:**

- **Error 500 (Internal Server Error):** Sistema reintenta automáticamente hasta 5 veces con backoff exponencial (1s, 2s, 4s, 8s, 16s).
- Si todos los reintentos fallan con 500: Plan permanece en estado `"sin autorización"` y la API devuelve 503 `Service Unavailable`.
- **Errores 400/404/otros:** Marcan el plan como `authorizationStatus="denegado"` inmediatamente sin reintentos.
- Si el POST a FAS falla sin respuesta (timeout/red), marca `denegado` y devuelve 500 con mensaje.
- Callback FAS actualiza estado final: `aprobado` si `state=ACCEPTED`, `denegado` en caso contrario.
- Envío automático de email con resultado de autorización y UPLAN adjunto (v2.1).

- **Geoawareness fallback:**

- Si el servicio no está configurado o no responde, el endpoint devuelve una respuesta mock y marca `serviceAvailable=false`.

- **Recuperación del pipeline de trayectorias:**

- Un plan puede re-ponerse en `en cola` desde la UI (PUT `/api/flightPlans`).
- `traj-assigner.js` reencola automáticamente planes procesados cuyo CSV asociado es menor a 2 KB (criterio implementado en BD).
- `machine-cleaner.js` borra todas las máquinas y reencola planes en procesando a `en cola`.

- **Validación de emails (v2.1):**

- Código de verificación: 6 dígitos, válido por 24 horas.
- Token de reset: válido por 1 hora desde generación.
- Errores de formato detectados con Zod schemas antes de persistencia.

7 Versionado y Control de Cambios

- **Versión del ICD:** v2.2.0.
- **Versión de la aplicación:** v2.2.0 SPIRIT (Sprints 1–5: 42 funcionalidades, proxy SSE de Geoawareness, visores 3D completos, gestión de usuario, sistema de contacto, restricciones EU, borradores U-Plan, visor Geoawareness 3D, correcciones UX y altimetría AGL en Geoawareness 3D).
- **Cambios en v2.2.0 SPIRIT:**
 - **Auth:** Eliminación de validación DNS en registro; caducidad de código de verificación a 3 min; doble-verificación safeguard.
 - **UI/UX:** Tema del sistema por defecto; toasts opacos; validación de archivo U-Plan en carga; tooltips `FieldHelp` en formularios.
 - **Perfil de usuario:** Campos `firstName`, `lastName`, `phone`; endpoints `GET/PATCH /api/user/profile`; pre-relleno automático en U-Plan form.
 - **Email:** Notificación al cancelar plan aprobado; email de confirmación en sistema de tickets.
 - **Plan Definition (ex Plan Generator):** Botón OK y Enter en edición de waypoints; botón “Añadir Waypoint” en sidebar; fix zoom reset SCAN; generación automática de volúmenes antes de descarga; navegación directa a Plan Authorization tras subir plan.
 - **FAS:** Coloreado selectivo de volúmenes conflictivos (intersección 2D); razón FAS parseada en lenguaje natural; sincronización del campo `state` del U-Plan con la decisión FAS.
 - **Restricciones regulatorias EU:** Tabla C0–C6 con MTOM y dimensión máxima; validación en formulario.
 - **Visores 3D CesiumJS:** `Cesium3DModal` (volúmenes operativos + OSM Buildings + slider 4D); `Denial3DModal` (primas 3D de geozonas conflictivas); `AuthorizationResult` (modal unificado 2D/3D); `Trajectory3DViewer` (dron animado + RAF + AGL unificado); InfoBox dark-theme fix en todos los visores.
 - **Contacto:** Página `/contact-us`; endpoint `POST /api/contact`; tickets `UPPS-AAAAMMDD-XXXX`.
 - **Renombrado de páginas:** Plan Generator → `/plan-definition`, Trajectory Generator → `/plan-authorization`.
 - **Proxy SSE Geoawareness:** Backend conecta `ws://` internamente y sirve SSE al navegador via `GET /api/geoawareness/stream/[uspaceId]`; elimina `mixed-content` en HTTPS; deprecia `NEXT_PUBLIC_GEOAWARENESS_SERVICE_IP`.
 - **Fix Cesium assets:** `postinstall/prebuild` copian directorio completo `Build/Cesium/` (incluyendo `Cesium.js`) a `public/cesium/`.
 - **Nuevos endpoints (Sprints 1–3):** `GET/PATCH /api/user/profile`, `POST /api/contact`, `GET /api/cesium/token`, `GET /api/geoawareness/stream/[uspaceId]`.
 - **Sprint 4 — Bug Fixes, UX Polish & Nuevas Funcionalidades:**
 - * Fix popup waypoint (OK cierra, Enter confirma); mejoras SCAN (Add Vertex, bounds efectivos, drag validado); segundo click en plan abre waypoints.

- * Fix color 4D (`ColorMaterialProperty/ConstantProperty`); fix polilínea trayectoria 3D (eliminado AMSL, unificado AGL).
 - * Nuevo `Geoawareness3DModal` con geozonas 3D extruidas, trayectoria y volúmenes.
 - * Auto-relleno email U-Plan desde perfil; sistema de borradores U-Plan (`uplanDraft` + endpoints CRUD).
 - * Eliminación de sección Flight Plan Details de Plan Definition (~840 líneas).
- **Sprint 5 — Correcciones de visores 3D, gestión de borradores y UX:**
- * **Fix preview waypoints en panel de plan seleccionado (F37):** Prop `onWaypointPreviewClick` propagada al componente `FlightPlanCard` en el panel lateral; click en plan seleccionado abre correctamente el modal de waypoints.
 - * **Fix renderizado continuo en Cesium3DModal (F38):** Se activa `viewer.scene.requestRenderMode = false` inmediatamente tras la creación del viewer, garantizando actualización continua de la escena para animaciones y carga de terreno.
 - * **Eliminación de polilínea de trayectoria en visores 3D (F39):** Reemplazada la polilínea continua de trayectoria en `Trajectory3DViewer` y `Geoawareness3DModal` por marcadores individuales de puntos: despegue (verde), aterrizaje (rojo) y waypoints intermedios (azul); se conservan etiquetas en despegue/aterrizaje.
 - * **Fix tema oscuro en InfoBox/widgets de Cesium (F40):** Inyección de CSS con `background: #1e1e1e !important; color: #e0e0e0 !important;` en el `<iframe>` de InfoBox y en los contenedores de widgets externos de los cinco visores CesiumJS (`Cesium3DModal`, `Denial3DModal`, `AuthorizationResultModal`, `Trajectory3DViewer`, `Geoawareness3DModal`).
 - * **Gestión de borradores U-Plan mejorada (F41):** Botón “Eliminar borrador” (rojo + icono de papelera) añadido al formulario `UplanFormModal`; botones “Actualizar” (teal) y “Guardar como borrador” (verde) con iconos diferenciados; eliminada opción de borrar borrador desde la página de Perfil (se centraliza en el formulario).
 - * **Auto-zoom de cámara a zona de vuelo en Geoawareness 3D:** La cámara en `Geoawareness3DModal` hace zoom al `BoundingSphere` de volúmenes de operación + trayectoria (en lugar de la totalidad de geozonas), mostrando el área de vuelo relevante al abrir el modal.
 - * **Fix altimetría AGL en volúmenes de operación en Geoawareness 3D (F42):** Los volúmenes de operación de U-Plan en `Geoawareness3DModal` se convierten de AGL a AMSL mediante `Cesium.sampleTerrainMostDetailed()` aplicado al centroide de cada polígono; se eliminan `heightReference: RELATIVE_TO_GROUND` y `extrudedHeightReference: RELATIVE_TO_GROUND` a favor de alturas absolutas AMSL calculadas explícitamente ($h_{AMSL} = h_{terreno} + h_{AGL}$); esto corrige la aparición de volúmenes a nivel del mar cuando las teselas de terreno no han cargado al inicializar el viewer.
- **Nuevos endpoints (Sprint 4):** `GET/POST /api/user/drafts`, `GET/PATCH/DELETE /api/user/drafts/[id]`.

• **Cambios en v2.1.0 SPIRIT:**

- **Sistema de emails completo:** Integración con MailerSend para verificación de email, reset de contraseña y notificaciones de autorización FAS.
- **Importación de UPLANs externos:** Soporte completo para drag-and-drop y upload de archivos .json con UPLANs pregenerados.
- **Selector de U-Space interactivo:** Modal con mapa para asignar U-Space a UPLANs externos antes de verificación de geoawareness.
- **Visualización de denegaciones:** Modal de mapa especializado para mostrar volúmenes conflictivos y razones de denegación.
- **Edición de waypoints SCAN:** Popups editables en mapas con campos para coordenadas (lat/lng).
- **Mejoras de tema UI:** Tema oscuro completo en popups Leaflet y componente ScanPatternGenerator.
- **Descarga de U-Plan:** Botón para descargar JSON del U-Plan procesado.
- **Reintentos FAS 500:** Lógica de reintentos con backoff exponencial para errores internos del FAS.
- **Correcciones de formato:** Conversión automática de scheduledAt a ISO-8601 con timezone para UPLANs externos.
- **Nuevos endpoints:**
 - * POST /api/auth/verify-email – Verificación de email con código
 - * POST /api/auth/request-password-reset – Solicitud de reset de contraseña
 - * POST /api/auth/reset-password – Reset de contraseña con token

- **Cambios en v2.0.0 SPIRIT:**

- **UI/UX Mejorado:** Sistema de temas dinámico (claro/oscuro) con CSS variables y persistencia.
- **Normalización de tiempos:** Trayectorias CSV normalizadas para iniciar en t=0s, eliminando offsets temporales.
- **Auto-regeneración de volúmenes:** Hook useVolumeRegeneration activo por defecto, verifica cada 30s planes procesados sin volúmenes.
- **Viewer de trayectorias mejorado:** Componente TrajectoryMapView con mapas Leaflet interactivos y análisis detallado.
- **Geoawareness avanzado:** Auto-centrado de mapa en trayectoria, marcadores reducidos (5/3px), overlays oscuros consistentes.
- **WebSocket optimizado:** Manejo de errores mejorado, no muestra errores durante conexión inicial, fallback híbrido.
- **Gestión de folders:** Drag-and-drop completo, sección de planes huérfanos, operaciones de renombrado y movimiento.
- **Timestamps dinámicos:** creationTime y updateTime del U-Plan se actualizan al momento del envío a FAS.
- **Workflow de 5 pasos:** Select → DateTime → Process → Geoawareness → Authorize con indicadores visuales.

- **Arquitectura modular:** Componentes organizados en `app/components/flight-plans/` con hooks especializados.
- **Endpoints optimizados:**
 - * `POST /api/flightPlans/regenerate-volumes`: Regeneración automática de volúmenes.
 - * `POST /api/flightPlans/[id]/reset`: Reset completo de plan a estado inicial.
 - * Mejoras en filtrado de planes procesados (`status='procesado', csvResult=1`).
- **Cambios en v1.0.0** (referencia histórica):
 - Migración de Geoawareness de HTTP polling a WebSocket para obtención de geozonas en tiempo real.
 - Nuevo endpoint WebSocket: `ws://<IP>/ws/gas/{USPACE_ID}`.
 - Nuevo formato de mensaje de geoawareness con 3 bloques estructurados.
 - Deprecación del endpoint HTTP `POST /geozones_searcher_by_volumes`.
 - Sistema de fallback híbrido (WebSocket primario, HTTP legacy como backup).
- **Control de cambios de interfaz:**
 - El sistema introduce un patrón de *API unificada* donde `/api/flightPlans` y `/api/csvResult` concentran operaciones individuales y bulk.
 - El endpoint `/api/flightPlans/[id]` se declara *deprecated* (placeholder) y no debe usarse para nuevas integraciones.
 - Para compatibilidad hacia atrás, se recomienda mantener campos opcionales (p.ej. `geoawarenessData`, `authorizationMessage`) sin romper respuestas existentes.
 - El endpoint HTTP de geoawareness está movido a `/api/deprecated/geoawareness-geozones` y solo debe usarse como fallback.
 - Nuevo sistema de regeneración automática opera de forma transparente sin intervención del usuario.

8 Integración con el Servicio Geoawareness

8.1 Arquitectura del Proxy SSE

A partir de la versión 2.2.0, la comunicación con el servicio Geoawareness ya no se realiza con una conexión WebSocket directa desde el navegador, sino a través de un proxy SSE alojado en el propio backend UPPS:

- **Motivación:** Una conexión `ws://` iniciada desde una página servida por HTTPS es bloqueada por los navegadores (política de mixed-content). El proxy SSE elimina esta restricción sin requerir TLS en el servicio Geoawareness.
- **Flujo:**

1. El navegador abre `GET /api/geoawareness/stream/{uspaceId}` como `EventSource` (SSE sobre HTTPS).
 2. El backend `Next.js` (runtime `Node.js`) abre `ws://<GEOAWARENESS_SERVICE_IP>/ws/gas/{uspaceId}` de forma interna usando la librería `ws`.
 3. Cada mensaje WebSocket recibido del servicio se retransmite al navegador como un evento SSE (tipo `data`).
 4. Si el navegador cierra la conexión SSE, el backend cierra el WebSocket interno para evitar fugas de recursos.
- **Evento SSE connected:** emitido cuando el WS interno se abre correctamente.
 - **Evento SSE error:** emitido si el WS interno reporta un error; el stream SSE se cierra.
 - **Evento SSE close:** emitido cuando el WS interno cierra; el stream SSE se cierra.
 - **Reconexión:** gestionada por el hook `useGeoawarenessWebSocket` con backoff exponencial sobre la conexión SSE.

8.2 Endpoint proxy SSE

- **URL:** `GET /api/geoawareness/stream/{uspaceId}`
- **Parámetro path:** `uspaceId` – Identificador del U-space (ej: `VLCUspace`)
- **Content-Type respuesta:** `text/event-stream`
- **Headers:** `Cache-Control: no-cache, no-transform`, `X-Accel-Buffering: no`
- **Autenticación:** ninguna (la autenticación se gestiona en el frontend; el endpoint es interno al mismo origen)

8.3 States de Conexión (hook)

El hook `useGeoawarenessWebSocket` gestiona los siguientes estados:

- **connecting:** Estableciendo conexión SSE al backend
- **connected:** El backend confirmó apertura del WS, recibiendo datos
- **disconnected:** Conexión SSE cerrada
- **error:** Error en la conexión SSE o WS interno

8.4 Reconexión Automática

El sistema implementa reconexión automática con backoff exponencial sobre la conexión SSE:

- Intentos máximos: 5
- Delays: `1s → 2s → 4s → 8s → 16s`
- Tras agotar intentos: Fallback a endpoint HTTP deprecado

8.5 Formato de Mensaje (inalterado)

El payload JSON de cada evento SSE es idéntico al que enviaba el antiguo WebSocket directo. Se estructura en 3 bloques principales:

8.5.1 Bloque 1: Campos de Control

Campo	Tipo	Descripción
uspace.identifier	String	Identificador único del U-space (ej: "VLCU-space")
timestamp	String (ISO-8601)	Momento de generación del mensaje

8.5.2 Bloque 2: Datos del U-space (uspace_data)

Objeto GeoJSON Feature que describe el área del U-space:

Campo	Tipo	Descripción
type	String	Siempre "Feature"
id	Integer	Identificador numérico del U-space
bbox	Array[Float]	Bounding box [minLon, minLat, maxLon, maxLat]
name	String	Nombre legible del U-space
source	String	Origen de los datos
geometry.type	String	Tipo de geometría ("Polygon", "MultiPolygon")
geometry.coordinates	Array	Coordenadas GeoJSON [[[lon,lat],...]]
geometry.verticalReference	Object	Límites verticales (ver tabla siguiente)
geometry.sub_type	String	Subtipo de geometría
geometry.radius	Float	Radio para geometrías circulares
properties.identifier	String	Identificador formal del U-space
properties.country	String	País (código ISO)
properties.type	String	Tipo de zona
properties.region	String	Región geográfica

8.5.3 Referencia Vertical (verticalReference)

Campo	Tipo	Descripción
upper	Float	Límite superior de altitud
upperReference	String	Referencia del límite superior ("AGL", "AMSL")
lower	Float	Límite inferior de altitud
lowerReference	String	Referencia del límite inferior ("AGL", "AMSL")
uom	String	Unidad de medida ("M" para metros)

8.5.4 Condiciones de Restricción (restrictionConditions)

Campo	Tipo	Descripción
uasClass	Array[String]	Clases de UAS permitidas (“C0”, “C1”, “C2”, etc.)
authorized	String	Tipo de autorización requerida
uasCategory	Array[String]	Categorías de operación permitidas
uasOperationMode	Array[String]	Modos de operación (“VLOS”, “BVLOS”)
maxNoise	Float	Nivel máximo de ruido permitido (dB)
specialOperation	String	Operaciones especiales permitidas
photograph	String	Restricciones de fotografía

8.5.5 Autoridad de Zona (zoneAuthority)

Campo	Tipo	Descripción
name	String	Nombre de la autoridad
service	String	Tipo de servicio
SiteURL	String	URL del sitio web
email	String	Email de contacto
phone	String	Teléfono de contacto
purpose	String	Propósito de la zona
intervalBefore	String	Tiempo de antelación requerido
contactName	String	Nombre del contacto

8.5.6 Aplicabilidad Limitada (limitedApplicability)

Campo	Tipo	Descripción
startDatetime	String (ISO-8601)	Inicio de validez de la restricción
endDatetime	String (ISO-8601)	Fin de validez de la restricción
schedule.day	Array[String]	Días de la semana aplicables
schedule.startTime	String	Hora de inicio (HH:MM)
schedule.startEvent	String	Evento de inicio
schedule.endTime	String	Hora de fin (HH:MM)
schedule.endEvent	String	Evento de fin

8.5.7 Bloque 3: Geozonas (geozones)

FeatureCollection GeoJSON con las geozonas asociadas al U-space. Cada Feature tiene la misma estructura que `uspace_data` (geometría, properties con `restrictionConditions`, `zoneAuthority`, `limitedApplicability`).

8.6 Ejemplo de Mensaje (payload SSE)

El campo `data` de cada evento SSE contiene el mismo JSON que enviaba el antiguo WebSocket:

```

{
  "uspace_identifier": "VLCUspace",
  "timestamp": "2026-02-03T10:00:00Z",
  "uspace_data": {
    "type": "Feature",
    "id": 1,
    "name": "Valencia U-space",
    "geometry": {
      "type": "Polygon",
      "coordinates": [[[-0.5, 39.4], [-0.3, 39.4], [-0.3, 39.6], [-0.5, 39.6], [-0.5, 39.4]]],
      "verticalReference": {
        "upper": 120, "upperReference": "AGL",
        "lower": 0, "lowerReference": "AGL", "uom": "M"
      }
    },
    "properties": {
      "identifier": "USP-ESP-VLC-01",
      "country": "ESP",
      "type": "uspace"
    }
  },
  "geozones": {
    "type": "FeatureCollection",
    "features": [
      {
        "type": "Feature",
        "geometry": { "type": "Polygon", "coordinates": [...] },
        "properties": {
          "identifier": "GZ_001",
          "type": "prohibited",
          "restrictionConditions": { "uasClass": ["C0", "C1"], ... },
          "zoneAuthority": { "name": "AESA", "email": "...", ... },
          "limitedApplicability": { "startDatetime": "...", ... }
        }
      }
    ]
  }
}

```

8.7 Sistema de Fallback

Si la conexión SSE falla después de 5 intentos de reconexión, el sistema activa automáticamente un fallback HTTP:

- **Endpoint fallback:** GET /api/deprecated/geoawareness-geozones
- **Fuente de datos:** lib/geozones/geozones_dataframe.geojson
- **Formato:** GeoJSON FeatureCollection (formato legacy)
- **Normalización:** El componente `geozone-normalizer.ts` convierte ambos formatos a una estructura común

Anexos

A. Variables de entorno (según `.env.example` y código)

- **DATABASE_URL:** conexión MySQL (ejemplo: `mysql://USER@HOST:3306/upps`).
- **FAS_API_URL:** URL completa del servicio FAS (incluye protocolo, host, puerto y path).

- `GEOAWARENESS_SERVICE_IP`: host:puerto del servicio geoawareness (servidor). Usado por el proxy SSE para abrir la conexión WebSocket interna.
- `GEOAWARENESS_ENDPOINT` (opcional): prefijo del path WebSocket (por defecto: `ws/gas/`).
- `NEXT_PUBLIC_GEOAWARENESS_SERVICE_IP`: **deprecado en v2.2.0**. Ya no es necesario; el navegador no abre WebSocket directamente.
- `MAILERSEND_API_KEY`: API key de MailerSend para envío de emails (verificación, reset, notificaciones).
- `MAILERSEND_SENDER_EMAIL` (opcional): Email del remitente (por defecto: `UPPS@sna-upv.com`).
- `MAILERSEND_SENDER_NAME` (opcional): Nombre del remitente (por defecto: `UPPS`).
- `NEXT_PUBLIC_APP_URL`: URL base de la aplicación para links en emails (por defecto: `http://localhost:3001`).
- `JWT_SECRET` (opcional, recomendado): secreto para JWT (si no existe, el sistema usa un valor por defecto en código).
- `NEXT_PUBLIC_GENERATE_RANDOM_UPLAN_DATA` (opcional): Si es `'true'`, genera datos aleatorios para campos faltantes en U-Plan (solo desarrollo).

B. Endpoints HTTP implementados (resumen)

- `GET /api/flightPlans?userId=<id>`
- `POST /api/flightPlans` (individual o bulk `items`; soporta importación UPLAN externo con `type="external_uplan"`)
- `PUT /api/flightPlans ({id,data} o bulk {ids,data} / {items})`
- `DELETE /api/flightPlans ({id} o bulk {ids})`; elimina también CSV asociado; envía cancelación a FAS si aprobado)
- `POST /api/flightPlans/regenerate-volumes` (v2.0: regeneración automática de volúmenes)
- `POST /api/flightPlans/<id>/reset` (v2.0: reset completo de plan a estado inicial)
- `POST /api/flightPlans/<id>/generate-volumes` (generación bajo demanda de volúmenes desde CSV)
- `GET /api/csvResult?id=<id>`
- `POST /api/csvResult` (bulk `{ids}`)
- `DELETE /api/csvResult ({id} o bulk {ids})`
- `POST /api/flightPlans/<id>/geoawareness` (verifica plan contra geozonas)
- `POST /api/flightPlans/<id>/uplan` (genera y envía U-Plan a FAS; detección automática de UPLAN externo; reintentos en error 500)

- PUT /api/fas/<externalResponseNumber> (callback de FAS; envía email de notificación con UPLAN adjunto)
- DELETE /api/uplan_cancellation/<externalResponseNumber> (cancelación de UPLAN aprobado en FAS)
- GET/POST /api/folders?userId=<id>
- GET/PUT/DELETE /api/folders/<id>
- POST /api/machines y GET /api/machines?name=<name>
- PUT /api/machines/<id>
- POST /api/auth/signup (registro con envío de email de verificación)
- POST /api/auth/login
- GET /api/user
- POST /api/auth/verify-email (v2.1: verificación de email con código de 6 dígitos o token)
- POST /api/auth/request-password-reset (v2.1: solicitud de reset con envío de email)
- POST /api/auth/reset-password (v2.1: reset de contraseña con token)
- GET /api/geoawareness/uspaces (proxy para lista de U-spaces)
- GET /api/geoawareness/stream/[uspaceId] (v2.2.0: proxy SSE de Geoawareness; retransmite WebSocket interno como SSE)
- GET /api/deprecated/geoawareness-geozones (fallback HTTP, deprecated)
- GET /api/user/drafts (v2.2.0 Sprint 4: lista borradores U-Plan del usuario)
- POST /api/user/drafts (v2.2.0 Sprint 4: crea un borrador U-Plan)
- GET /api/user/drafts/[id] (v2.2.0 Sprint 4: obtiene un borrador específico)
- PATCH /api/user/drafts/[id] (v2.2.0 Sprint 4: actualiza nombre o datos de un borrador)
- DELETE /api/user/drafts/[id] (v2.2.0 Sprint 4: elimina un borrador)

C. Endpoints de streaming

- GET /api/geoawareness/stream/{uspaceId} – Proxy SSE de Geoawareness (v2.2.0). El backend abre internamente `ws://<GEOAWARENESS_SERVICE_IP>/ws/gas/{uspaceId}` y retransmite los mensajes al navegador como eventos SSE.
- **(Servidor interno, no expuesto):** `ws://<GEOAWARENESS_SERVICE_IP>/ws/gas/{USPACE_ID}` – Solo accedido por el backend.

D. Flujo de alto nivel (texto)

1. El usuario abre **Plan Definition** (/plan-definition) y selecciona un U-space.
2. El cliente web conecta al proxy SSE backend /api/geoawareness/stream/{USPACE_ID} que a su vez abre ws://.../ws/gas/{USPACE_ID} para recibir geozonas en tiempo real.
3. El usuario define waypoints sobre el mapa, visualizando las geozonas como referencia.
4. El usuario sube planes: POST /api/flightPlans con fileContent y status="sin procesar".
5. Tras subir, la app navega automáticamente a **Plan Authorization** (/plan-authorization?selectedPlanId=...) y auto-selecciona el plan importado.
6. **Workflow de 5 pasos** (Plan Authorization):
 - (a) **Select:** Usuario selecciona un plan de vuelo de la lista.
 - (b) **DateTime:** Usuario establece fecha/hora programada (scheduledAt).
 - (c) **Process:** Usuario procesa el plan (marca como "en cola" mediante PUT /api/flightPlans).
 - (d) **Geoawareness:**
 - Sistema regenera volúmenes automáticamente si faltan (cada 30s).
 - Usuario revisa U-Plan generado y verifica contra geozonas via WebSocket.
 - Usuario visualiza trayectoria procesada en mapa interactivo.
 - (e) **Authorize:** Usuario envía U-Plan a FAS para autorización.
7. traj-assigner.js asigna planes "en cola" a máquinas "Disponibile" y marca el plan "procesando".
8. Los workers externos generan el CSV con normalización de tiempos y actualizan BD (csvResult y estado del plan).
9. Check Geoawareness: El usuario visualiza la trayectoria contra geozonas actualizadas via WebSocket en tiempo real.
10. Envío a autorización: POST /api/flightPlans/<id>/uplan (con timestamps actualizados) y callback FAS a PUT /api/fas/<externalResponseNumber>.
11. Sistema regenera volúmenes automáticamente si el plan se resetea o pierde sus volúmenes.

E. Arquitectura de Componentes (v2.0)

La aplicación se estructura en componentes modulares organizados por funcionalidad:

Componentes principales

- **FlightPlansUploader**: Interfaz principal del **Plan Authorization** con workflow guiado.
- **PlanGenerator / Plan Definition**: Generador de planes de vuelo con mapa interactivo y waypoints (página `/plan-definition`).
- **ProcessingWorkflow**: Indicador visual del workflow de 5 pasos.
- **FolderList**: Gestión de carpetas con drag-and-drop.
- **FlightPlanCard**: Tarjeta de plan con operaciones individuales.
- **DateTimePicker**: Selector de fecha/hora con soporte UTC.

Componentes de visualización

- **Trajectory3DViewer**: Visor 3D de trayectoria con dron animado (Cesium), altitudes AGL.
- **GeoawarenessViewer**: Visualizador de trayectoria sobre geozonas con SSE en tiempo real.
- **Cesium3DModal**: Visor 3D de volúmenes de operación U-Plan con slider temporal 4D.
- **AuthorizationResultModal**: Modal unificado de resultado de autorización (2D + 3D).
- **Denial3DModal**: Visor 3D de denegación con geozonas conflictivas.
- **Geoawareness3DModal**: Visor 3D de geozonas extruidas con trayectoria y volúmenes (v2.2.0 Sprint 4).
- **WaypointMapModal**: Modal para preview de waypoints en plan.
- **UplanViewModal**: Visualizador de volúmenes de operación en mapa.
- **UplanFormModal**: Formulario de edición de campos U-Plan (con tooltips `FieldHelp`).
- **GeozoneLayer**: Capa de geozonas con colores por tipo y opacidad configurable.

Hooks especializados

- **useFlightPlans**: Gestión de planes con polling opcional (5s).
- **useFolders**: Gestión de carpetas.
- **useGeoawarenessWebSocket**: Conexión SSE con reconexión automática (v2.2.0: proxy backend).
- **useVolumeRegeneration**: Regeneración automática de volúmenes (30s).
- **usePolling**: Polling genérico con control de errores.

- `useTheme`: Gestión del tema claro/oscuro con persistencia.
- `useToast`: Notificaciones toast con estados y retry.

Componentes UI reutilizables (v2.2.0)

- `FieldHelp`: Icono ? con tooltip al click; explica acrónimos de aviación en formularios U-Plan.

Modelo de datos adicional (v2.2.0 Sprint 4)

- `uplanDraft`: Modelo Prisma para borradores de U-Plan. Campos: `id` (autoincremental), `userId` (FK → user), `name` (varchar), `draftData` (JSON), `createdAt`, `updatedAt`. Índice en `userId`; cascada en eliminación del usuario.

Sistema de temas (v2.0)

- CSS variables dinámicas: `--bg-primary`, `--surface-primary`, `--text-primary`, etc.
- Cambio instantáneo sin recarga de página.
- Persistencia en `localStorage`.
- Soporte para `MutationObserver` en componentes dinámicos (header, footer).
- Variables de estado: `--status-success-bg`, `--status-error-bg`, etc.

Novedades en V2.4.0

Plan Activation — Flujo completo de activación de vuelo

- **Nuevos campos en `flightPlan`**: `activationStatus`, `activatedAt`, `activationMessage`, `termsAcceptedAt`, `lastActivationAttempt`. Estados posibles de `activationStatus`: *no activable*, *activable*, *activando*, *autorizado_despegue*, *denegado_activacion*, *ventana_pasada*.
- `GET /api/flightPlans/[id]/terms`: Proxy autenticado al FAS. Devuelve los T&C del plan aprobado bajo demanda. Requiere `authorizationStatus = aprobado` y `externalResponseNumber` presente.
- `POST /api/flightPlans/[id]/activate`: Valida ventana ± 60 s respecto a `scheduledAt`, cooldown de 5 s entre intentos, y `termsAccepted = true` en el cuerpo. Llama a `GET /activation/[externalResponseNumber]` en el FAS con timeout de 10 s. Persiste resultado en `activatedAt` / `activationStatus`.
- `GET /api/flightPlans/active`: Lista planes aprobados con `scheduledAt` en $[now - 1h, now + 24h]$.
- `GET /api/flightPlans/history`: Lista paginada de planes históricos (activados, ventana pasada, denegados/retirados). Parámetros `limit` (máx. 100) y `offset`.

- **Página /plan-activation:** Sección “Vuelos activables” (filtrado ± 60 s en cliente con tick de 1 s) y sección “Historial” colapsable con paginación.
- **ActivateFlightModal:** Carga T&C del FAS en `JsonViewerSections`, exige aceptación explícita, ejecuta la activación y muestra “PUEDE DESPEGAR” durante 60 s tras respuesta positiva.

Novedades en V2.3.0

Correcciones y estabilidad

- **Eliminación del cierre de sesión por inactividad (`fix(auth)`):** Se elimina completamente la lógica de *inactivity timeout* (10 min) del hook `useAuth.ts`. La sesión ahora solo expira cuando el *refresh token* caduca; no se cierran sesiones activas por ausencia de eventos de ratón/teclado.
- **Corrección del error SSR de Leaflet (`fix(ui)`):** El componente `DenialMapModal` importaba `react-leaflet` directamente, provocando el error `window is not defined` en el servidor. Se aísla la lógica del mapa en `DenialMapContent.tsx` con importaciones dinámicas (`ssr: false`), siguiendo el patrón ya presente en `TrajectoryMapView` y `GeoawarenessViewer`.

Propuesta de trayectoria alternativa SCRS

- **Parser de mensajes SCRS (`lib/scrs.ts`):** Nuevo módulo que extrae la alternativa de trayectoria del campo `scr_dispatch` del `authorizationMessage` del FAS. Guarda: `sent=true`, `status_code=200`, `response.status='success'`, segmentos no vacíos. Convierte coordenadas GeoJSON [*lon, lat, alt*] a {*lat, lon, alt*} y concatena los `route_path` de todos los segmentos, deduplicando puntos en las costuras.
- **Builder QGC reutilizable (`lib/qgc-plan.ts`):** Extracción de la función de generación de planes `.plan` (`QGroundControl`) a una librería independiente (`buildQgcPlan`). El componente `PlanGenerator.tsx` se refactoriza para llamar a este helper. Permite reutilización desde el endpoint de aceptación.
- **Modal comparativo de trayectorias (`AlternativeTrajectoryModal`):** Nuevo componente SSR-safe que muestra la trayectoria actual (polilínea azul) y la propuesta por el SCRS (polilínea verde discontinua) sobre un mapa Leaflet. Incluye `FitBoundsHandler`, estadísticas de waypoints y botones *Rechazar* / *Aceptar alternativa*.
- **Endpoint de aceptación (POST `/api/flightPlans/id/accept-alternative`):** Cuando el usuario acepta la alternativa, el endpoint: (1) valida la alternativa SCRS; (2) construye la lista de waypoints (puntos SCRS + waypoint de aterrizaje final a 0 m); (3) regenera `fileContent` en formato QGC `.plan`; (4) actualiza `uplan.flightDetails.waypoints` y vacía `operationVolumes`; (5) ejecuta una transacción Prisma que borra el `csvResult` y reinicia el plan a `status='sin procesar'` preservando `scheduledAt` y los metadatos del U-Plan.
- **Integración en UI (`FlightPlansUploader`):** El modal de resultado de autorización muestra un botón *Ver alternativa* (verde) solo cuando el plan está denegado

y el mensaje contiene un `scr_dispatch` válido. El flujo de aceptación llama al nuevo endpoint, muestra toast de confirmación y refresca la lista de planes.

- **Compatibilidad del flujo de reautorización:** El plan reprocesado sigue el pipeline estándar (*sin procesar* → *en cola* → *procesado* → autorización FAS → email de resultado), sin cambios en los componentes existentes.